

1. How to Explain HC2L and LazyHC2L (Simple but Technical)

◆ What HC2L Really Is

HC2L stands for **Hierarchical Cut 2-Hop Labeling**.

It's a **labeling algorithm**, not a **search-based pathfinding algorithm**.

What That Means

- Traditional algorithms like **Dijkstra** or **A* search** the map **every time** you ask for a route. They start from the origin and expand node by node until they reach the destination.
→ This is slow for large road networks.
- **HC2L does not search.**
Instead, before any query happens, it **precomputes labels** for every node. Each label stores the shortest distance to a few key “hub nodes.”
When you ask for a route later, it doesn't explore the graph — it **just looks up** those labels.

How HC2L Works (Conceptually)

Step 1: Preprocessing (One-Time Setup)

1. The road network is divided into smaller parts (using “hierarchical cuts”).
2. Each node stores **shortcut distances** to important *hub* nodes. These hubs connect local regions together.
3. This process builds a **label index**, which looks something like:

Label(A): [(H1, 2.3 km), (H5, 5.1 km)]

Label(B): [(H1, 3.0 km), (H7, 2.6 km)]

Step 2: Query (When You Ask for a Route)

1. To find the distance from **A** → **B**, HC2L simply looks at the **common hubs** between A and B.
For example:

Common hub: H1

Distance = $\text{dist}(A, H1) + \text{dist}(H1, B)$

2. The system picks the smallest of these combinations — that's the **shortest distance**.

✓ Result: The query takes **microseconds**, because there's **no search, no graph expansion, no loops**.

Why HC2L Works for Navigation

Although HC2L only returns the **shortest distance**, your **LazyHC2L tool adds a path reconstruction step**:

- **HC2L**: gives the numeric distance between A and B.
- **Your system**: backtracks the label intersections to get the actual list of nodes.
- **Frontend**: visualizes that sequence as the route on the map.

So:

“HC2L provides the *brain* (the mathematical computation),
and our tool provides the *body* — the map visualization and dynamic behavior.”

Do We Still Use Dijkstra or Not?

-  **Not for main routing.**
Once HC2L has built its labels, **Dijkstra is no longer needed** for normal queries.

HC2L answers instantly by label intersection.

-  **Yes, only in rare cases (optional).**
Dr. Henning Köhler explained that HC2L uses a small Dijkstra-like search **only for tiny disconnected parts of the map** called “*dangling trees*.”
These are exceptions, not the main logic.

So in your system:

“We use labeling (HC2L) for all route queries,
and Dijkstra is only used inside HC2L’s preprocessing or for small cleanup cases
— never for the actual query.”

How LazyHC2L Improves HC2L

HC2L is very fast but **not dynamic** — if the map changes (a road closes, traffic rises), it normally has to rebuild all labels.

Your **LazyHC2L** fixes this using two mechanisms:

1. Lazy Updates

- When a road changes, you **don’t rebuild everything**.
- You just **mark affected nodes as “dirty.”**
- The system only recomputes those nodes **when a user actually queries through them**.

 Example:

Road X closed → Nodes 45–48 marked dirty

If next query touches Node 46 → recompute labels for that subgraph

If not → skip (save time)

This is why it’s called “Lazy” — it waits until the update is truly needed.

2. 📊 Threshold-Based Triggering

Not all disruptions matter equally.

- Compute an **Impact Score** = $f(\Delta w) \times f_{jam} \times f_{closure}$.
- If it's **below τ (the threshold)** → do nothing (lazy mark only).
- If it's **above τ** → recompute labels immediately.

👉 Example:

Minor slowdown (Impact = 0.3) → Skip update

Major accident (Impact = 0.8) → Trigger update now

This keeps updates fast, selective, and realistic for live road conditions.

🔄 HC2L vs LazyHC2L Comparison

Feature	HC2L	LazyHC2L
Handles static networks	✅	✅
Handles dynamic disruptions	❌ Full rebuild required	✅ Selective lazy repair
Query speed	⚡ Ultra fast	⚡ Ultra fast
Update cost	🐢 Expensive	🧠 Efficient (only affected parts)
Memory use	Moderate	Lower (prunes unnecessary updates)

✅ TL;DR (Defensible Summary for Panel)

“HC2L is not a pathfinding algorithm — it’s a labeling-based query framework. Instead of searching, it precomputes all necessary information so queries become instantaneous.

Our tool extends HC2L by adding Lazy Updates and Threshold Triggers, allowing it to adapt to real-world road disruptions efficiently — it only recalculates what’s needed, when it’s needed.”



2. Programmer Requirements (Simplified)

Core Goal

Build a working prototype that:

- Shows how the route changes when a disruption happens.
- Logs and visualizes what LazyHC2L does internally (which nodes were updated, which were skipped).

Architecture Overview

1. **Data Input:** Load Quezon City OSM road network
2. **Disruption Detector:** Detect closed or congested roads
3. **Impact Evaluator:** Compute **ImpactScore**
4. **Threshold Module:** Compare against τ
5. **Lazy Update System:** Mark nodes dirty and update only on query
6. **Query Engine:** Compute route using labels (HC2L logic)
7. **Frontend:** Map + logs showing why route changed